100 More Advanced Frontend Tasks

> Become a Frontend Expert with These Comprehensive Challenges

---

PART A: Form Challenges (25 Tasks)

Section 1: Basic to Intermediate Forms (10 Tasks)

Task 1: Contact Form with Validation States

Goal: Build a contact form with visual validation feedback

- Fields: Name, Email, Phone, Subject, Message
- Real-time validation with visual states (success/error borders)
- Character counter for message (max 500)
- Submit button disabled until valid
- Success animation on submit
- Clear form after submission

TypeScript: Define `ValidationState` type

```
type ValidationState = 'idle' | 'validating' | 'valid' | 'invalid'

interface FieldState {
  value: string
  state: ValidationState
  error?: string
}
```

Task 2: Login Form with "Remember Me"

Goal: Authentication form with persistence option

- Fields: Email/Username, Password
- "Remember Me" checkbox stores credentials
- Show/hide password toggle

- "Forgot Password" link
- Social login buttons (Google, GitHub)
- Rate limiting after 3 failed attempts

Libraries: `react-hook-form`, `zod`

Task 3: Registration Form with Password Requirements

Goal: Sign up form with password strength indicator

- Fields: Username, Email, Password, Confirm Password
- Password requirements checklist:
  - ✓ Min 8 characters
  - ✓ One uppercase
  - ✓ One lowercase
  - ✓ One number
  - ✓ One special character
- Visual strength meter (weak/medium/strong)
- Terms and conditions checkbox

Task 4: Profile Settings Form

Goal: Edit user profile with image upload

- Pre-fill from user data
- Avatar upload with crop/resize
- Fields: Name, Bio, Location, Website, Social Links
- Character limits with counters
- Save button shows only if changes made
- Cancel reverts to original data

Task 5: Newsletter Subscription Form

Goal: Simple form with email validation and preferences

- Email input with format validation
- Frequency preference (Daily/Weekly/Monthly)
- Category interests (multi-select checkboxes)
- GDPR consent checkbox
- Animated success state
- Already subscribed detection

Task 6: Survey Form with Multiple Question Types

Goal: Dynamic survey with various input types

- Question types: Text, Radio, Checkbox, Scale (1-10), Dropdown
- Conditional questions based on previous answers
- Progress indicator
- Save draft functionality
- Required vs optional questions
- Submit validation

TypeScript:

```
type QuestionType = 'text' | 'radio' | 'checkbox' | 'scale' | 'dropdown'

interface Question {
  id: string
  type: QuestionType
  text: string
  options?: string[]
  required: boolean
  condition?: { questionId: string; answer: string }
}
```

Task 7: Search Form with Filters

Goal: Advanced search with multiple filter options

- Search input with autocomplete suggestions
- Category filter dropdown
- Date range picker
- Price range slider
- Sort by dropdown
- Active filters display with remove option
- Clear all filters button

Task 8: Feedback/Rating Form

Goal: Collect user feedback with star rating

- Star rating component (1-5 stars)
- Emoji rating option (😞😐😊😄🤩)

- Category selection (Bug, Feature, General)
- Description textarea
- Screenshot upload option
- Anonymous submission toggle

Task 9: Job Application Form

Goal: Multi-section application form

- Personal Information section
- Education history (add multiple)
- Work experience (add multiple)
- Skills selection with proficiency level
- Resume/CV file upload
- Cover letter textarea
- References (add multiple)

Task 10: Booking/Reservation Form

Goal: Date and time slot booking

- Service/Room selection
- Date picker (disable past dates)
- Available time slots display
- Number of guests/participants
- Special requests textarea
- Contact information
- Booking confirmation preview

Section 2: Advanced Form Patterns (10 Tasks)

Task 11: Address Autocomplete Form

Goal: Form with Google Places autocomplete

Libraries: `@react-google-maps/api` or `use-places-autocomplete`

- Address search with autocomplete
- Auto-fill: Street, City, State, ZIP, Country
- Manual override option

- Map preview of selected address
- Save multiple addresses
- Default address selection

Task 12: Credit Card Payment Form

Goal: Secure payment form with card detection

Libraries: `react-credit-cards-2`

- Card number with formatting (4444 4444 4444 4444)
- Auto-detect card type (Visa, MasterCard, Amex)
- Expiry date picker (MM/YY)
- CVV with tooltip explanation
- Card preview component
- Luhn algorithm validation

Task 13: Dynamic Form Builder

Goal: Create forms via drag-and-drop

Libraries: `dnd-kit`, `react-hook-form`

- Drag field types to canvas
- Configure field properties (label, placeholder, required)
- Reorder fields via drag
- Preview mode
- Export form schema as JSON
- Import and render from JSON

Task 14: Inline Edit Table Form

Goal: Edit table data inline

- Click cell to edit
- Tab to move to next cell
- Enter to save, Escape to cancel
- Validation per cell
- Bulk edit mode
- Undo/redo changes
- Save all changes button

Task 15: Dependent Dropdowns Form

Goal: Cascading select inputs

- Country → State → City dropdowns
- Load options dynamically via API
- Show loading state per dropdown
- Reset child dropdowns when parent changes
- Allow "Other" with text input
- Pre-select from saved data

Task 16: Form with Repeatable Sections

Goal: Add/remove dynamic form sections

- "Add Another" button for sections
- Minimum 1 section required
- Maximum limit (e.g., 5 sections)
- Remove button per section
- Drag to reorder sections
- Collapse/expand sections
- Copy section functionality

Task 17: Scheduled Form Submission

Goal: Submit form at a scheduled time

- Fill form and schedule for later
- Date/time picker for submission
- Preview scheduled submissions
- Edit before submission time
- Cancel scheduled submission
- Email notification on submit
- Recurring submission option

Task 18: Form with Conditional Pricing

Goal: Dynamic price calculation form

- Product/service selection
- Quantity input
- Add-ons checkboxes with prices

- Discount code input
- Real-time price calculation
- Tax calculation by region
- Order summary section

Task 19: Multi-Language Form

Goal: Form with i18n support

Libraries: `react-i18next`, `i18next`

- Language switcher
- RTL support for Arabic/Hebrew
- Translated labels and placeholders
- Localized validation messages
- Date/number format by locale
- Persist language preference

Task 20: Signature Capture Form

Goal: Digital signature input

Libraries: `react-signature-canvas`

- Draw signature on canvas
- Clear and redo options
- Touch/stylus support
- Save as image (PNG/SVG)
- Legal agreement checkbox
- Date stamp on signature

Section 3: Form UX Enhancements (5 Tasks)

Task 21: Smart Form with Auto-Save

Goal: Never lose form data

- Auto-save to localStorage every 30 seconds
- Auto-save on field blur
- "Draft saved" indicator
- Restore draft on return

- Clear draft on successful submit
- Conflict resolution if data changed

Task 22: Accessible Form with ARIA

Goal: Fully accessible form implementation

- Proper label associations
- Error announcements for screen readers
- Focus management on errors
- Skip to error link
- Form landmarks and headings
- High contrast mode support
- Keyboard navigation test

Task 23: Form Analytics Dashboard

Goal: Track form completion metrics

- Field-level analytics (time spent, errors)
- Abandonment tracking
- Completion rate calculation
- Heatmap of problematic fields
- A/B test different form versions
- Export analytics report

Task 24: AI-Assisted Form Filling

Goal: Smart suggestions for form fields

Libraries: OpenAI API

- Smart autocomplete suggestions
- Auto-fill from clipboard detection
- Grammar/spell check for text fields
- Address standardization
- Phone number formatting by country
- Email domain suggestions

Task 25: Form Comparison Tool

Goal: Side-by-side form data comparison

- Load two versions of form data
- Highlight differences
- Field-by-field comparison
- Merge changes option
- Version history
- Rollback to previous version

PART B: UI Component Challenges (25 Tasks)

Section 4: Core UI Components (10 Tasks)

Task 26: Accordion Component

Goal: Expandable content sections

- Single or multiple open mode
- Animated height transitions
- Keyboard navigation (arrow keys)
- Nested accordions support
- Disabled state
- Custom icons

TypeScript:

```
interface AccordionItem {
  id: string
  title: string
  content: React.ReactNode
  disabled?: boolean
}
```

Task 27: Breadcrumb Navigation

Goal: Show navigation hierarchy

- Auto-generate from route
- Truncate long paths with ellipsis
- Last item is current (non-clickable)

- Custom separators (/, >, →)
- Responsive: show only last 2 on mobile
- Structured data for SEO

Task 28: Card Component Library

Goal: Versatile card component system

- Variants: basic, elevated, outlined, interactive
- Header, body, footer sections
- Image placement options (top, left, background)
- Loading skeleton state
- Selectable card mode
- Card group layout

Task 29: Chip/Tag Component

Goal: Compact element for labels

- Variants: filled, outlined, soft
- Colors: primary, secondary, success, error, warning
- Deletable with × button
- Clickable/selectable mode
- Avatar/icon prefix
- Size variants (sm, md, lg)

Task 30: Tooltip Component

Goal: Informational popup on hover

- Positions: top, right, bottom, left
- Smart positioning (flip if overflow)
- Delay on show/hide
- Arrow pointer
- Rich content support
- Click to keep open mode

Libraries: Consider `@floating-ui/react`

Task 31: Progress Indicators

Goal: Various progress UI components

- Linear progress bar
- Circular progress (donut)
- Step progress (wizard)
- Animated transitions
- Determinate and indeterminate modes
- Custom colors and sizes
- Success/error states

Task 32: Avatar Component

Goal: User representation component

- Image avatar with fallback
- Initials fallback from name
- Status indicator (online, offline, busy)
- Size variants
- Avatar group with overlap
- "+X more" for overflow
- Click to view profile

Task 33: Badge/Notification Dot

Goal: Small count or status indicator

- Position: top-right, top-left, etc.
- Max count display (99+)
- Dot mode (no number)
- Animated on change
- Colors for different states
- Standalone and wrapped modes

Task 34: Divider Component

Goal: Visual separation between content

- Horizontal and vertical
- With centered text/icon
- Dashed, dotted, solid styles
- Custom thickness
- Margin/spacing options

- Semantic HTML (hr)

Task 35: Empty State Component

Goal: Display when no data available

- Illustration/icon
- Title and description
- Action button (e.g., "Create First Item")
- Variants: no data, error, no results
- Customizable content
- Animated illustration option

Section 5: Advanced UI Components (10 Tasks)

Task 36: Command Palette (Cmd+K)

Goal: Keyboard-driven action search

Libraries: `cmdk` or custom

- Global keyboard shortcut (Cmd/Ctrl+K)
- Search through actions/pages
- Recent searches
- Keyboard navigation
- Grouped results
- Nested commands
- Custom actions per item

Task 37: Data Table with All Features

Goal: Feature-rich data table

Libraries: `@tanstack/react-table`

- Column sorting (multi-column)
- Global and column-specific filters
- Pagination with page size options
- Row selection (single/multi)
- Column pinning (left/right)
- Column resizing

- Row expansion
- Export to CSV/Excel

Task 38: Tree View Component

Goal: Hierarchical data display

- Expand/collapse nodes
- Checkbox selection (parent-child sync)
- Keyboard navigation
- Search/filter nodes
- Lazy loading for large trees
- Drag and drop reorder
- Custom node rendering

Task 39: Timeline Component

Goal: Chronological event display

- Vertical and horizontal modes
- Alternating sides layout
- Custom icons per event
- Date grouping (by day/month/year)
- Animated on scroll into view
- Expandable event details
- Load more functionality

Task 40: Kanban Board

Goal: Column-based task management

Libraries: `dnd-kit`

- Multiple columns (Todo, In Progress, Done)
- Drag cards between columns
- Drag to reorder within column
- Add new cards
- Edit card inline
- Column management (add, rename, delete)
- WIP limits per column

Task 41: Calendar Component

Goal: Full-featured calendar view

Libraries: `date-fns`, `react-big-calendar` or custom

- Month, week, day views
- Event display with colors
- Click date to add event
- Drag event to reschedule
- Recurring events
- Mini calendar for navigation
- Today button

Task 42: Image Gallery/Lightbox

Goal: Image viewing with zoom

Libraries: `react-photo-album`, `yet-another-react-lightbox`

- Grid/masonry layout
- Click to open lightbox
- Previous/next navigation
- Zoom and pan
- Slideshow mode
- Thumbnail strip
- Download option

Task 43: Color Picker Component

Goal: Select colors with various inputs

- Hue/saturation picker
- RGB sliders
- HEX input
- HSL input
- Recent/saved colors
- Eyedropper tool
- Gradient picker option

Task 44: Rich Text Editor

Goal: WYSIWYG text editing

Libraries: `tiptap`, `slate`, or `lexical`

- Bold, italic, underline, strikethrough
- Headings (H1-H6)
- Lists (ordered, unordered, checklist)
- Links with preview
- Images with upload
- Code blocks with highlighting
- Tables
- Mentions (@user)

Task 45: Notification Center

Goal: In-app notification panel

- Bell icon with unread count
- Dropdown panel with notifications
- Mark as read (single/all)
- Delete notifications
- Group by date
- Different types (info, success, warning)
- Link to related content
- Real-time updates

Section 5: Interactive UI Patterns (5 Tasks)

Task 46: Onboarding Tour

Goal: Guide new users through UI

Libraries: `react-joyride`, `driver.js`

- Step-by-step highlights
- Tooltips pointing to elements
- Progress indicator
- Skip and back options
- Don't show again checkbox
- Trigger on first visit
- Resume incomplete tour

Task 47: Sticky Table Header/Column

Goal: Fixed headers while scrolling

- Sticky header row
- Sticky first column
- Shadow on scroll for depth
- Handle both horizontal and vertical scroll
- Responsive behavior
- Works with virtual scrolling

Task 48: Collapsible Sidebar

Goal: Expandable navigation sidebar

- Expand/collapse toggle
- Icons only when collapsed
- Hover to temporarily expand
- Nested menu items
- Active item highlighting
- Pin/unpin option
- Keyboard shortcut toggle

Task 49: Split Pane/Resizable Panels

Goal: Adjustable panel layout

Libraries: `react-resizable-panels`

- Drag handle to resize
- Minimum/maximum sizes
- Horizontal and vertical splits
- Nested splits
- Collapse panel option
- Save layout preference
- Reset to default

Task 50: Floating Action Button (FAB) Menu

Goal: Expandable action button

- Primary FAB button
- Expand to show secondary actions
- Speed dial style animation
- Position options (bottom-right typical)
- Hide on scroll option
- Label tooltips
- Mobile-friendly touch targets

PART C: Third-Party Package Integration (25 Tasks)

Section 6: Data Visualization (8 Tasks)

Task 51: Dashboard Charts

Goal: Interactive dashboard with multiple charts

Libraries: `recharts` or `chart.js` with `react-chartjs-2`

- Line chart for trends
- Bar chart for comparisons
- Pie/Donut for distribution
- Area chart for volumes
- Responsive sizing
- Tooltips on hover
- Legend with toggle
- Export as image

Task 52: Real-Time Data Chart

Goal: Live updating chart

Libraries: `recharts` , WebSocket or polling

- Streaming data display
- Auto-scroll to latest
- Pause/resume live updates
- Zoom and pan
- Time range selector
- Data point count limit
- Smooth animations

Task 53: Geographic Map Visualization

Goal: Data on world/country map

Libraries: `react-simple-maps`, `d3-geo`

- Choropleth map (colored regions)
- Markers/pins on locations
- Zoom to region
- Tooltip on hover
- Legend for scale
- Click region for details
- Custom projections

Task 54: Organizational Chart

Goal: Hierarchical org structure

Libraries: `react-organizational-chart`, `d3-hierarchy`

- Tree layout of employees
- Expand/collapse branches
- Search and highlight
- Click for employee details
- Drag to reorganize
- Export as image/PDF
- Print-friendly view

Task 55: Gantt Chart

Goal: Project timeline visualization

Libraries: `gantt-task-react`, `frappe-gantt`

- Task bars on timeline
- Dependencies (arrows)
- Drag to reschedule
- Resize to change duration
- Milestone markers
- Progress percentage
- Multiple view modes (day/week/month)

Task 56: Heatmap Calendar

Goal: GitHub-style contribution graph

Libraries: `react-calendar-heatmap`, `cal-heatmap`

- Year view with daily cells
- Color intensity by value
- Tooltip with details
- Click for day details
- Multiple years navigation
- Custom color scales
- Legend

Task 57: Network Graph

Goal: Node and edge visualization

Libraries: `react-force-graph`, `vis-network`, `cytoscape`

- Nodes with labels
- Edges connecting nodes
- Force-directed layout
- Zoom and pan
- Click node for details
- Highlight connected nodes
- Add/remove nodes dynamically

Task 58: Sankey Diagram

Goal: Flow visualization

Libraries: `recharts`, `d3-sankey`

- Flow between stages
- Width proportional to quantity
- Hover to highlight path
- Click for breakdown
- Multiple levels
- Color coding by category
- Interactive filtering

Section 7: Media & File Handling (7 Tasks)

Task 59: Video Player

Goal: Custom video player with controls

Libraries: `react-player`, `video.js`, `plyr-react`

- Play/pause, seek
- Volume control with mute
- Fullscreen toggle
- Playback speed options
- Picture-in-picture
- Keyboard shortcuts
- Quality selection
- Subtitles/captions

Task 60: Audio Player/Podcast Player

Goal: Audio playback with waveform

Libraries: `wavesurfer.js`, `react-h5-audio-player`

- Play/pause, seek
- Waveform visualization
- Playback speed (0.5x - 2x)
- Skip 15s forward/back
- Playlist support
- Download option
- Sleep timer

Task 61: Document Viewer

Goal: View PDFs and documents

Libraries: `react-pdf`, `@react-pdf-viewer/core`

- Render PDF pages
- Page navigation
- Zoom controls
- Search in document
- Download and print

- Thumbnail sidebar
- Annotations

Task 62: Image Editor

Goal: Basic image editing capabilities

Libraries: `react-image-crop`, `tui-image-editor`

- Crop with aspect ratios
- Rotate and flip
- Brightness/contrast
- Filters (grayscale, sepia)
- Draw/annotate
- Resize
- Undo/redo
- Export in different formats

Task 63: File Manager

Goal: Dropbox-like file browser

Libraries: Custom with API

- Grid and list views
- Breadcrumb navigation
- Create folder
- Upload files (drag or click)
- Rename, delete, move
- Multi-select with Shift/Ctrl
- Context menu (right-click)
- File preview

Task 64: Markdown Editor/Previewer

Goal: Side-by-side markdown editing

Libraries: `react-markdown`, `@uiw/react-md-editor`

- Split view (editor | preview)
- Toolbar for formatting
- Live preview

- Syntax highlighting for code
- Image upload/embed
- Export as HTML/PDF
- Table of contents generation

Task 65: QR Code Generator/Scanner

Goal: Generate and scan QR codes

Libraries: `qrcode.react`, `react-qr-scanner`

- Generate QR from text/URL
- Customize size and colors
- Logo in center option
- Download as image
- Camera scanner
- Parse scanned content
- History of scanned codes

Section 8: Communication & Real-Time (5 Tasks)

Task 66: Real-Time Chat Interface

Goal: WhatsApp-like chat UI

Libraries: `socket.io-client`, `pusher-js`

- Message list with timestamps
- Send text messages
- Typing indicator
- Read receipts
- Message reactions
- Image/file attachments
- Online status
- Message search

Task 67: Video Call Interface

Goal: Zoom-like video meeting UI

Libraries: `twilio-video`, `daily-co`, `livekit`

- Local and remote video
- Mute/unmute audio
- Camera on/off
- Screen sharing
- Participant grid view
- Speaker view
- Chat sidebar
- Raise hand feature

Task 68: Collaborative Document Editor

Goal: Google Docs-like real-time editing

Libraries: `yjs`, `socket.io`, `tiptap`

- Multiple cursors
- Real-time sync
- User presence indicators
- Conflict resolution
- Version history
- Comments and suggestions
- Offline support with sync

Task 69: Notification Push System

Goal: Browser push notifications

Libraries: Web Push API, Service Workers

- Permission request
- Desktop notifications
- Click to open app
- Notification actions
- Badge count
- Sound notification option
- Subscription management

Task 70: Activity Feed

Goal: Social media-like feed

- Infinite scroll

- Different activity types (post, like, comment, follow)
- Time ago formatting
- User avatars and links
- Expandable threads
- Reactions (like, love, etc.)
- Share functionality

Section 9: E-commerce Components (5 Tasks)

Task 71: Product Card Grid

Goal: E-commerce product listing

- Image with hover gallery
- Quick view modal
- Add to cart button
- Wishlist toggle
- Price with sale/discount
- Rating stars
- Stock status
- Compare checkbox

Task 72: Shopping Cart

Goal: Full cart functionality

- Item list with images
- Quantity +/- buttons
- Remove item
- Price calculations
- Promo code input
- Shipping estimation
- Save for later
- Clear cart

Task 73: Product Filters

Goal: Advanced filtering for product lists

- Price range slider

- Category checkboxes
- Brand multi-select
- Size selector
- Color swatches
- Rating filter
- In stock only toggle
- Apply/clear buttons

Task 74: Checkout Flow

Goal: Multi-step checkout process

Libraries: `react-hook-form`, `stripe`

- Cart review
- Shipping address
- Shipping method selection
- Payment method
- Order review
- Place order
- Order confirmation
- Email receipt

Task 75: Product Reviews System

Goal: Review and rating functionality

- Star rating input
- Pros/cons lists
- Photo upload
- Helpful votes
- Sort reviews (newest, helpful)
- Filter by rating
- Response from seller
- Verified purchase badge

PART D: Performance & Optimization (25 Tasks)

Section 10: React Performance (10 Tasks)

Task 76: Memo Optimization Lab

Goal: Practice React.memo effectively

- Build component with expensive renders
- Add console.log to track renders
- Implement React.memo
- Custom comparison function
- Measure before/after with React DevTools
- Document when memo helps/hurts

Task 77: useMemo/useCallback Workshop

Goal: Optimize computed values and callbacks

- Create expensive calculation scenario
- Implement useMemo for computation
- Use useCallback for event handlers
- Track render performance
- Dependency array best practices
- Identify unnecessary memoization

Task 78: Virtual List Implementation

Goal: Build virtualized list from scratch

- Calculate visible items range
- Dynamic item heights
- Overscan for smooth scrolling
- Scroll to index functionality
- Handle resize
- Benchmark: render 100,000 items smoothly

Task 79: Code Splitting Strategies

Goal: Optimize bundle with splitting

Tools: Vite, webpack-bundle-analyzer

- Route-based splitting
- Component-based splitting
- Split vendor bundles

- Preload critical routes
- Prefetch on hover
- Measure bundle reduction

Task 80: Image Optimization Pipeline

Goal: Optimal image loading strategy

Libraries: `next/image` or custom

- Lazy loading with Intersection Observer
- Responsive images (srcset)
- WebP/AVIF with fallback
- Blur placeholder
- Priority loading for above-fold
- CDN integration

Task 81: State Management Optimization

Goal: Prevent unnecessary renders from state

- Split state into smaller pieces
- Use context selectors
- Implement Zustand selectors
- Avoid prop drilling renders
- State colocation principles
- Benchmark render counts

Task 82: Form Performance Optimization

Goal: Handle large forms efficiently

Libraries: `react-hook-form`

- Uncontrolled vs controlled inputs
- Field-level subscriptions
- Avoid re-render on every keystroke
- Array field optimization
- Lazy validation
- 1000 field form benchmark

Task 83: Animation Performance

Goal: 60fps animations

Libraries: `framer-motion`, CSS transforms

- Use transform over position
- will-change property
- Avoid layout thrashing
- Request animation frame
- GPU acceleration
- Performance timeline analysis

Task 84: Event Handler Optimization

Goal: Efficient event handling patterns

- Debounce search inputs
- Throttle scroll handlers
- Event delegation
- Passive event listeners
- Remove listeners on unmount
- Measure event processing time

Task 85: Suspense and Concurrent Features

Goal: Leverage React 18+ features

- Suspense for data fetching
- useTransition for slow updates
- useDeferredValue for expensive renders
- Suspense boundaries
- Error boundaries
- Loading states

Section 11: Web Performance (10 Tasks)

Task 86: Core Web Vitals Dashboard

Goal: Monitor and improve CWV

Tools: web-vitals library, Lighthouse

- Measure LCP (Largest Contentful Paint)
- Measure FID (First Input Delay)
- Measure CLS (Cumulative Layout Shift)
- Real user monitoring
- Identify bottlenecks
- Implement fixes
- Before/after comparison

Task 87: Service Worker Caching

Goal: Offline-first application

Libraries: Workbox

- Cache static assets
- Cache API responses
- Stale-while-revalidate strategy
- Offline fallback page
- Background sync
- Cache invalidation
- Storage management

Task 88: Critical CSS Extraction

Goal: Inline above-the-fold CSS

Tools: Critical, PurgeCSS

- Extract critical CSS
- Inline in head
- Async load remaining CSS
- Remove unused CSS
- Measure render blocking
- FCP improvement

Task 89: Resource Hints Implementation

Goal: Preload and prefetch resources

- `<link rel="preload">` for critical assets

- `<link rel="prefetch">` for next page
- `<link rel="preconnect">` for APIs
- DNS prefetch
- Module preload
- Font preload strategies

Task 90: Network Request Optimization

Goal: Minimize network overhead

- Request deduplication
- Request batching
- GraphQL query optimization
- Response compression (gzip, brotli)
- HTTP/2 multiplexing
- Connection pooling
- Cache headers

Task 91: Font Loading Optimization

Goal: Prevent font flash issues

- `font-display: swap`
- Subset fonts (remove unused glyphs)
- WOFF2 format
- Preload critical fonts
- System font fallback
- Variable fonts
- Font loading API

Task 92: Third-Party Script Management

Goal: Control third-party impact

- Defer non-critical scripts
- Async loading patterns
- Partytown for worker
- Script loading priority
- Remove unused scripts
- Self-host critical third-party
- Monitor third-party impact

Task 93: Memory Leak Detection

Goal: Find and fix memory leaks

Tools: Chrome DevTools Memory tab

- Identify leak sources
- Clean up subscriptions
- Clear timers on unmount
- Weak references
- Event listener cleanup
- Heap snapshot analysis
- Memory profiling

Task 94: Bundle Analysis and Reduction

Goal: Minimize JavaScript bundle

Tools: webpack-bundle-analyzer, source-map-explorer

- Analyze bundle composition
- Find duplicate dependencies
- Tree shaking verification
- Replace heavy libraries
- Dynamic imports
- Compression comparison

Task 95: Rendering Performance Audit

Goal: Eliminate render bottlenecks

Tools: Chrome DevTools Performance tab

- Record and analyze timeline
- Identify long tasks
- Reduce main thread work
- Optimize JavaScript execution
- Avoid forced synchronous layouts
- Compositor layers optimization

Section 12: Testing & Quality (5 Tasks)

Task 96: Component Unit Testing

Goal: Comprehensive component tests

Libraries: `vitest`, `@testing-library/react`

- Render testing
- User interaction testing
- Async behavior testing
- Mock API calls
- Snapshot testing
- Accessibility testing
- 80%+ coverage target

Task 97: Integration Testing

Goal: Test component interactions

Libraries: `vitest`, `msw`

- Multi-component flows
- Form submission flow
- Navigation testing
- State management testing
- API integration testing
- Error scenario testing

Task 98: End-to-End Testing

Goal: Full user flow testing

Libraries: `playwright` or `cypress`

- Complete user journeys
- Cross-browser testing
- Visual regression testing
- Performance testing
- Mobile viewport testing
- CI/CD integration

Task 99: Error Boundary Implementation

Goal: Graceful error handling

- React Error Boundary component
- Fallback UI display
- Error reporting to service
- Retry mechanism
- Different boundaries per section
- Development vs production behavior

Task 100: Performance Testing Suite

Goal: Automated performance checks

Tools: Lighthouse CI, web-vitals

- Set performance budgets
- Automated Lighthouse runs
- Bundle size checks
- Core Web Vitals thresholds
- CI/CD integration
- Performance regression alerts
- Historical tracking

📊 Task Categories Summary

Category	Task Numbers	Count
Forms	1-25	25
UI Components	26-50	25
Third-Party Packages	51-75	25
Performance	76-100	25

🎯 Difficulty Progression

Beginner (Start Here)

Tasks: 1-5, 26-30, 51-53, 76-77

Intermediate

Tasks: 6-15, 31-40, 54-60, 78-85

Advanced

Tasks: 16-20, 41-45, 61-70, 86-95

Expert

Tasks: 21-25, 46-50, 71-75, 96-100

📅 Suggested Learning Path

Month 1: Forms Mastery

- Week 1: Tasks 1-7 (Basic Forms)
- Week 2: Tasks 8-14 (Intermediate Forms)
- Week 3: Tasks 15-20 (Advanced Forms)
- Week 4: Tasks 21-25 (Form UX)

Month 2: UI Components

- Week 1: Tasks 26-35 (Core Components)
- Week 2: Tasks 36-42 (Advanced Components)
- Week 3: Tasks 43-50 (Interactive Patterns)
- Week 4: Build a component library

Month 3: Third-Party Integration

- Week 1: Tasks 51-58 (Data Viz)
- Week 2: Tasks 59-65 (Media)
- Week 3: Tasks 66-70 (Real-Time)
- Week 4: Tasks 71-75 (E-commerce)

Month 4: Performance & Testing

- Week 1: Tasks 76-82 (React Performance)
- Week 2: Tasks 83-89 (Web Performance)

- Week 3: Tasks 90-95 (Advanced Optimization)
- Week 4: Tasks 96-100 (Testing)

✅ Per-Task Checklist

- [ ] Understand requirements completely
- [ ] Research libraries and patterns
- [ ] Implement core functionality
- [ ] Add TypeScript types
- [ ] Handle loading/error states
- [ ] Make it responsive
- [ ] Add keyboard accessibility
- [ ] Write at least 3 tests
- [ ] Optimize performance
- [ ] Document usage
- [ ] Deploy and share

🔧 Recommended Tech Stack

```
{
  "framework": "Next.js 14+ / React 18+",
  "language": "TypeScript",
  "styling": "Tailwind CSS + shadcn/ui",
  "forms": "react-hook-form + zod",
  "state": "Zustand / TanStack Query",
  "testing": "Vitest + Playwright",
  "animation": "Framer Motion",
  "charts": "Recharts",
  "tables": "@tanstack/react-table"
}
```

🚀 Pro Tips

1. **Don't just code** - Understand WHY each pattern exists
2. **Read the docs** - Library documentation is your friend
3. **Check bundle size** - Run `bundlephobia.com` before installing
4. **Test on mobile** - 60%+ of users are on mobile

5. **Accessibility first** - Use semantic HTML and ARIA
6. **Performance budgets** - Set limits before you start
7. **Code review** - Share and get feedback

> **Remember:** The goal isn't to rush through all 100 tasks. It's to deeply understand each concept and build muscle memory for good practices. Quality over quantity! 💪